# CS780 Capstone Project: OBELIX: the Warehouse Robot

Magesvar V R
251010068
magesvar25@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

**Abstract**

The OBELIX warehouse robot task is addressed here, in which a circular differential drive robot tries to attach a piece of garbage to the boundary of a constrained environment. The robot is presented with a discrete action space and 18-bit sensory information. The severely under-informed observation space makes the problem a partially observable Markov decision process (POMDP). To address this problem, various reinforcement learning techniques were evaluated, such as REINFORCE, Vanilla Policy Gradient, and Proximal Policy Optimization under policy-based algorithms and DQN and DDQN in value-based algorithms. Among the traditional variants of this algorithm, techniques such as LSTM policy and value networks augmented with a four-class heuristic mode signal and mode-aware differential logic bias, a state-aware reward shaper, and a deterministic stuck escape finite state machine, Option-Critic learning etc., are tried. All training episodes were run on the basis of curriculum learning, where different configurations were tried.

## 1 Competition Result

**Codabench Username:** m_251010068
**Final leaderboard rank (Test Phase):** 36
**Leaderboard rank – Level 1 (static):** 67
**Leaderboard rank – Level 2 (blinking):** 84
**Leaderboard rank – Level 3 (moving + blinking):** 98

Table 1: Number of valid submissions per phase.

| Phase | Dev submissions | Best score |
|---|---|---|
| Level 1 – Static | *6* | *-1982.5* |
| Level 2 – Blinking | *2* | *-1991.00* |
| Level 3 – Moving+Blinking | *2* | *-190009.26 (note: +1845.6 (2nd highest without wall))* |
| Test Phase (all levels) | 2 | *-1827.81* |

## 2 Introduction and Problem Description

The environment presents a robotic manipulation problem in which a circular differential drive robot with a discrete action space and 18-bit sensory information in a 2D arena tries to attach to a grey box and push the box to the boundary wall and also at the same time recover from wall collisions. The 18-bit sensory information consists of 16 sonar bits (o[0:15]- encoding eight directional beams each with near and far range), one infrared proximity bit (O[16] - encoding any detection by the infrared sensor), and one stuck flag bit (set when a robot does not move for several steps consecutively). As mentioned earlier, the robot has a discrete action space with dimension 5 (L22, L45, F2, R45, R22) representing:

- Left turns of 22° and 45° respectively
- Forward step
- Right turns of 22° and 45° respectively

The task is a partially observable Markov decision process due to the fact that the robot receives only a binary snapshot at each step. It cannot directly observe its own heading, the absolute position of the box, or whether a sonar corresponds to the target box or a wall obstacle. Meaning that identical observations can represent two different things in the world, creating the problem of perceptual aliasing. Looking at this problem, the first solution that one could search for is using a long short-term memory (LSTM) layer in the value and policy networks to integrate the observations across time.

There are three environmental difficulty levels.

1. **Level-1** - places static blocks at a random position.
2. **Level-2** - places blinking (periodically disappearing) static blocks at random positions.
3. **Level-3** - dynamic blinking blocks moving at random trajectories deployed at random positions.

The simulation environment is designed with two distinct operational modes to evaluate the robustness of the navigation algorithm:

**Mode 1: Wall-Obstacles** In this configuration, static wall obstacles are strategically inserted into the workspace.

**Mode 2: Open Workspace** This mode consists of a clean environment without any wall obstacles.

The Wall-Obstacles problem adds the main problem of perceptual aliasing, making it significantly harder for the agents to solve the problem. Perceptual aliasing is basically the problem of ambiguity in perception in this mode. This arises because both the walls and the box are detected by the sonars, and the infrared sensors sends sensory information as observations, so this makes the agent confused.

# 3 Approach and Model Evolution

## 3.1 Baseline/initial attempts:

For approaching this problem, the first agent that was selected was a Double Deep Q-Network (DDQN) using a feedforward architecture and experience replay. After fine-tuning the agent, it was observed that the agent consistently collapsed into a pure rotation policy, accumulating large negative penalties for getting stuck rather than exploring. From analyzing this failure, it was very evident that it is a credit assignment problem, and this occurred due to the sparse success reward structure. However, in an attempt to solve using the same approach, the exploration was increased and tried, but in the later stage of the policy, it was observed that the agent again collapsed into a rotation policy. However, the use of value-based algorithm such as DDQN led to a time-consuming training process, and hence the focus was shifted towards policy-based algorithms.

In policy-based algorithms, Vanilla Policy Gradient, REINFORCE and PPO agents were tried, and it was observed that they also collapsed to the same rotation policy. Realizing the difficulty of the problem after further research, the idea of using LSTM in the policy networks was implemented, which prevented immediate entropy collapse, but the agent still failed to learn a useful policy.

However, these attempts also turned out to be useless as the 18-bit raw sensory information contains no information about the robot's heading or time since the robot was last seen. Without structured features, the LSTM has no meaning.

### 3.2 Key Design Decisions and Evolution:

This section includes only some of the prominent methods attempted to solve the problem, not all. Early experiments with PPO-LSTM revealed another class of instability without careful constraint that the forward action probability would decay to zero and the policy would become trapped in a rotation-only local optimum. To bypass this problem, forward bias was implemented in the training phase with the help of three complementary fixes:

1. Forward **FW** probability floor was introduced directly in the forward pass of the policy network, clamping $Pr[FW] < \delta_{FW}$ and re-normalizing without any in-place tensor operations.
2. The network's output bias was initialized with $b_{FW} = 1.0$ To give the forward action warm-up advantage
3. A global probability floor. $\delta_{global}$ was applied after softmax so that no action is ever completely suppressed.

But it was observed that particular seeds in which the boxes were right in front of the bot made the bot move forward in a large fraction, which caused the agent to be highly forward-biased. To solve this problem, three other fixes were enforced.

1. A logit temperature Parameter t was added to scale all logits by force of max. This widened the initial distribution and slowed entropy decay.
2. Per episode, advantage normalization was applied together with advantage Asymmetric advantage clamp (min -2.0, max +5.0). This allows large gradients in the positive direction, whereas there are small gradients in the negative direction. Further, for the first 100 episodes, the negative floor was tightened to -1 to protect the randomized policy from getting into catastrophic early updates.
3. KL penalty and a diversity loss were added to the PPO objective to further prevent collapse.

Even though the above methods improve the policy from just being a rotational policy to a forward-moving policy. The perceptual aliasing problem still exists, and hence the agent kept attempting to attack the walls.

### 3.3 Heuristic Mode Augmentation

In an attempt to solve the problem of perceptions in observations, a four-class heuristic mode signal was derived from the raw observation and was fed into the observation space to the agent to perform better. Given $O_t \in \{0,1\}^{18}$, the mode $m_t$ is computed by the function **get_mode :**

$$m_t = \begin{cases} \text{Unstuck} & \text{if } o_{17} = 1 \\ \text{Push} & \text{else if } o_{16} = 1 \\ \text{Align} & \text{else if } \exists\, i \in \{0,\ldots,15\} : o_i = 1 \\ \text{Search} & \text{otherwise} \end{cases} \tag{1}$$

The board is then converted into a four-dimensional one-hot array $m_t \in \{0,4\}^4$ and and concatenated into the raw observation

$$\mathbf{s}_t = \big[\underbrace{o_t}_{18}, \underbrace{\text{onehot}(m_t)}_{4}\big] \in \mathbb{R}^{22}.$$

This partially resolves the perceptual problem, and this solved the environment without a wall. However, the agent gets confused between align, unstuck, and push states, and collapses into getting stuck against the wall trying to push the box. Crucially, the mode is only used as an input feature. No actions were forced or overridden based on MDP. Preserving full PPO validity.

### 3.4 Reward Shaping and MDP Design

To facilitate efficient policy convergence and prevent the agent from becoming trapped in sub-optimal local minima, a state-dependent reward shaping function $R(s, a, s')$ was implemented. The total reward at each timestep is categorized based on the agent's operational mode, defined as:

$$R_{total} = \begin{cases} R_{search} & \text{if } s \in \mathcal{S}_{search} \\ R_{align} & \text{if } s \in \mathcal{S}_{align} \\ R_{push} & \text{if } s \in \mathcal{S}_{push} \\ R_{stuck} & \text{if } s \in \mathcal{S}_{stuck} \end{cases} \tag{2}$$

#### 3.4.1 Behavioral Hierarchy and Components

The reward shaper incentivizes transitions through the following behavioral states:

- **Search and Exploration:** In the absence of sensor hits, turns are rewarded ($+0.12$) to encourage scanning, while blind forward movement is penalized ($-0.50$) to prevent aimless drifting into boundaries.
- **Alignment Control:** Once a target is detected via sonar, the reward is tied to the angular error $\theta_{err}$ derived from the active sonar units. We define the centering reward as $R_{align} = \eta(1 - \frac{|\theta_{err}|}{\pi})$, where $\eta$ is a scaling constant. This encourages the policy to minimize the boresight error before attempting a push.
- **Contact and Sustained Interaction:** A sparse *Contact Bonus* ($+0.50$) is awarded upon initial IR sensor activation. To maintain engagement, a *push streak* mechanism adds a cumulative bonus ($+0.02 \times n$) for each consecutive step contact is maintained without the agent entering a stuck state.
- **Stuck Recovery Logic:** To discourage "ramming" behaviors, any forward action that results in a stuck state receives a heavy penalty ($-0.40$). Conversely, the agent receives a marginal positive reward ($+0.04$) for executing turns while stuck, guiding the gradient toward the Finite State Machine (FSM) escape maneuvers.

### 3.5 Curriculum Learning

Training directly on full task complexity leads to poor outcomes due to sparse rewards and early collapse to rotation-only policies. Both agents, therefore, employ curriculum strategies that progressively increase difficulty.

**Wall-obstacle curriculum** For the first 300 episodes, the environment is instantiated without walls, allowing the agent to learn the core locate-align-push behaviour unobstructed. At episode 300, walls are permanently enabled for the remainder of training. The environment seed for episode $e$ is fixed as seed $+ e$, ensuring deterministic but diverse arena layouts.

**Alternating block curriculum (DDQN).** Training episodes are divided into blocks of $B = 10$ consecutive episodes, alternating between wall-free and wall-present environments. For the global episode counter $e_g$, walls are active when $\lfloor e_g/B \rfloor$ is odd. This interleaved schedule exposes the Q-network to both variants throughout training, preventing overfitting to either regime and providing a natural form of domain randomisation.

**Exploration annealing (PPO-LSTM).** Four exploration parameters — forced-forward rate $r_{\mathrm{FW}}$, $\varepsilon$-greedy rate, SEARCH-mode random bonus, and UNSTUCK forward penalty — decay linearly over training:

$$x(e) = x_{\mathrm{init}}\Big(1 - \frac{e}{E}\Big) + x_{\mathrm{final}}\,\frac{e}{E}. \tag{3}$$

High initial values guide the agent toward productive behaviours early; as they decay the learned policy progressively takes full control.

# 4 Experiments and Implementation Details

All experiments were run on CPU (Intel Core i7, Windows 11). Training used the Adam optimizer with separate learning rates for policy and value networks and gradient clipping was done. Also, curriculum learning was implemented. The wall curriculum parameter explains the curriculum learning. Heuristic PPO-LSTM, and DQN hyperparameters are listed below and these performed better compared to the other methods.

Table 2: Hyperparameter comparison between Heuristic PPO-LSTM and DDQN architectures.

| Hyperparameter | Heuristic PPO-LSTM | DDQN |
|---|---|---|
| *Core Algorithm* | PPO (Clipped Surrogate) | Double DQN (DDQN) |
| Network Architecture | FC $\rightarrow$ LSTM $\rightarrow$ FC | FC $\rightarrow$ FC $\rightarrow$ FC (3-layer MLP) |
| Hidden Units | 128 per layer | 128 per layer |
| Observation Dim | 22 (Augmented with Mode) | 18 (Raw) |
| Discount Factor ($\gamma$) | 0.99 | 0.98 |
| Optimizer | Adam ($10^{-4}$ policy, $10^{-3}$ value) | Adam ($10^{-4}$) |
| *Training Dynamics* | | |
| Exploration Strategy | Logit Temp / Entropy Decay | $\varepsilon$-Greedy ($1.0 \rightarrow 0.05$) |
| Advantage Estimation | GAE ($\lambda = 0.95$) | N/A |
| Batch Size | Episode-based | 64 transitions |
| Target Update Freq | N/A (On-policy) | 1000 steps |
| Gradient Clipping | 0.5 (Global Norm) | N/A |
| Replay Buffer Size | N/A | 50,000 transitions |
| *Reward & Constraints* | | |
| Advantage Clamp | $[-2.0, +5.0]$ | N/A |
| Entropy Coef ($\beta$) | $0.10 \rightarrow 0.02$ (Linear) | N/A |
| Probability Floor ($\delta$) | 0.05 (Global), 0.10 (FW) | N/A |
| Rotation Penalty ($\rho$) | 0.01 | 0.01 |
| Wall Curriculum | Switch at ep. 500 | Alternating (10 ep. blocks) |
| Max Episode Steps | 200 | 300 |

# 5 Results

Table 3 compares mean cumulative reward across model variants on the development phases.

Table 3: Mean cumulative reward per model variant across difficulty levels (development phase). Higher is better.

| Model / Variant | Level 1 | Level 2 | Level 3 | Notes |
|---|---|---|---|---|
| DQN baseline | -1982.5 | - | - | Feedforward, no memory |
| VPG feedforward | -1982.1 | - | [score] | No memory |
| VPG-LSTM | -1982.1 | - | - | Episode memory, raw obs |
| PPO-LSTM (raw obs) | - | -1991 | - | No mode, no shaping |
| PPO-LSTM + heur. | - | - | [540 (no wall) / -190k] | Final model with mode-aware heuristics |

Below is one of the plots which represents heuristic modes entered during the training process without wall

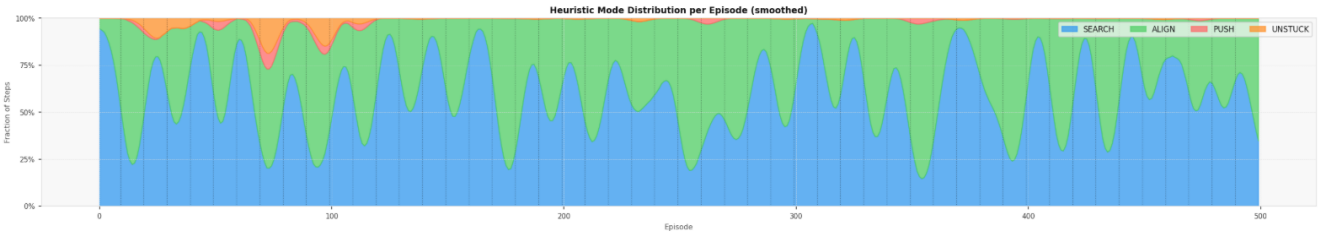


Figure 1: Heuristic Mode Example

# 6 Error Analysis and Discussion

## 6.1 Perceptual Ambiguity and Wall Obstacles

The primary challenge in the environment is the *perceptual aliasing* between the target box and the environmental boundaries. Because the 16-bit sonar array returns identical signatures for both entities, the agent cannot determine if a sensor hit represents a goal-related object or a static obstacle based on a single observation. This was the main failure mode across all phases.

To resolve this, the policy must integrate multi-modal signals:

**Sensor Fusion:** The agent utilizes the STUCK flag and IR contact sensors as ground-truth disambiguate. A sonar hit without subsequent IR contact or a STUCK event is eventually treated as a wall.

**Heading Problem:** in this case the problem arises because the agent does not have any information about the heading of the robot so when the agent attaches to the box, it does not know where the box has attached to it. Hence it tries to unwedge in a different orientation compared to where the box is, receiving a negative reward.

### 6.2 Limitations and Future Work

The binary, low-resolution observation space remains the fundamental bottleneck for robust navigation. Future work will focus on:

1. **World Models:** Implementing a recurrent world model [1] to predict future sensor states, allowing the agent to "imagine" the difference between a moving box and a fixed wall.

2. **Feature Augmentation:** Adding relative heading or odometry to break the symmetry of the environment.

## 7 Conclusion

This project dived into deep reinforcement learning for a POMDP robotic manipulation task with severe impoverishment in sensory data. The main finding is that the raw 18-bit observation is insufficient for a purely reactive policy as this causes the problem of perceptual aliasing and makes the agent unable to learn. With the heuristic implementation in the above method, this provided the LSTM with structured subtask context without any hard-coded actions; however the problem of perception still arose when the environment had walls. The above is one of the main methods that was implemented. Along with this there were other methods tried, such as a world model [2] [1] for learning how observation and the actual environment are related to solve the perception problem and also an option critic [3] framework was tried for making the heuristic model better.

## LLM Usage Declaration

- **Which Large Language Model (LLM) did you use?**
  Claude (Anthropic) – claude.ai, model Claude Sonnet 4.6.

- In which part did you use the LLM, and how did it help you?

| Part | Used LLM? (Yes/No) | Purpose of Using LLM |
|---|---|---|
| Understanding concepts | Yes | Explaining POMDP, PPO, LSTM, GAE theory |
| Ideation / brainstorming | Yes | Mode signal design, exploration strategies |
| Debugging small code snippets | Yes | Tensor shape errors, in-place autograd bugs |
| Plotting / visualization code | No | — |
| Grammar / writing style | YeS | Used Dictation (WisprFlow) |
| Report structure / section ideas | No | — |

- Did you refer to any other sources or websites apart from the LLM and lecture slides?
  Yes, mentioned in References.

- Apart from any LLM usage, what was your own contribution?
  All implementation ideas where done by students, reading research papers and ideation of multiple solutions for problems. Ai was used to code many logic provided by the student. However, the main code was taken from lecture slides and some changes where made to code as and when required.

**Link for Video:** https://drive.google.com/file/d/1pLGPCrTK0J6uS4Vh-4kShoAxbCTGB1oq/view?usp=shar

**Github Repository Link:** https://github.com/Mag-27/CS780-OBELIX-251010068-MAGESVAR-V-R.git

## References


[1] D. Hafner *et al.*, "Mastering diverse domains through world models," *arXiv preprint arXiv:2301.04104*, 2023.

[2] D. Ha and J. Schmidhuber, "World models," in *Advances in Neural Information Processing Systems*, 2018.

[3] P.-L. Bacon, J. Harb, and D. Precup, "The option-critic architecture," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2017.